

Programming

factory method pattern

Factory Method Patternは、オブジェクトを使用するコードで直接newして生成するのではなく、オブジェクトの生成を別のクラスやメソッドに任せるpatternである

Factory Method Patternを使うと、オブジェクトを使用するコードで、newを使って具象クラスを直接生成すると、結合度が高くなる

そして、オブジェクトの種類が追加・変更された場合、修正範囲が広がる可能性がある

上位クラスではオブジェクトを使用する処理を定義して、実際にどのオブジェクトを生成するかはサブクラスで決める

```

package fmp;

public abstract class PizzaStore {

    public Pizza orderPizza(String type) {

        Pizza pizza = createPizza(type);

        pizza.prepare();
        pizza.bake();
        pizza.cut();

        return pizza;
    }

    protected abstract Pizza createPizza(String type);
}

```

まず、factoryメソッドを作る

createPizza()がFactory Methodになる

PizzaStoreでは、どのPizzaを生成するかを決めない

PizzaStoreを継承するサブクラスでcreatePizza()を実装する

```

public abstract class Pizza {

    protected String name;

    public void prepare() {
        System.out.println(name + " 준비");
    }

    public void bake() {
        System.out.println(name + " 굽기");
    }

    public void cut() {
        System.out.println(name + " 자르기");
    }

    public String getName() {
        return name;
    }
}

```

```

public class NYCheesePizza extends Pizza {

    public NYCheesePizza() {
        name = "뉴욕 치즈 피자";
    }
}

public class ChicagoCheesePizza extends Pizza {

    public ChicagoCheesePizza() {
        name = "시카고 치즈 피자";
    }
}

```

Pizzaは抽象クラスにして、各PizzaクラスはPizzaを継承する

```

public class NYPizzaStore extends PizzaStore {

    @Override
    protected Pizza createPizza(String type) {

        if (type.equals("cheese")) {
            return new NYCheesePizza();
        }

        if (type.equals("pepperoni")) {
            return new NYPepperoniPizza();
        }

        throw new IllegalArgumentException("피자가 존재하지 않음");
    }
}

```

NYPizzaStoreでは、new NYCheesePizza()を生成する

```

public class ChicagoPizzaStore extends PizzaStore {

    @Override
    protected Pizza createPizza(String type) {

        if (type.equals("cheese")) {
            return new ChicagoCheesePizza();
        }

        if (type.equals("pepperoni")) {
            return new ChicagoPepperoniPizza();
        }

        throw new IllegalArgumentException("존재하지 않는 피자입니다.");
    }
}

```

ChicagoPizzaStoreでは、new ChicagoCheesePizza()を生成する

```

public class Main {

    public static void main(String[] args) {

        PizzaStore nyStore = new NYPizzaStore();
        PizzaStore chicagoStore = new ChicagoPizzaStore();

        Pizza pizza1 = nyStore.orderPizza("cheese");

        System.out.println();
        System.out.println("주문 완료: " + pizza1.getName());

        System.out.println("-----");

        Pizza pizza2 = chicagoStore.orderPizza("pepperoni");

        System.out.println();
        System.out.println("주문 완료: " + pizza2.getName());

    }
}

```

```

뉴욕 스타일 치즈 피자 준비
뉴욕 스타일 치즈 피자 굽기
뉴욕 스타일 치즈 피자 자르기

주문 완료: 뉴욕 스타일 치즈 피자
-----
시카고 스타일 페퍼로니 피자 준비
시카고 스타일 페퍼로니 피자 굽기
시카고 스타일 페퍼로니 피자 자르기

주문 완료: 시카고 스타일 페퍼로니 피자

```

このようにPizzaStoreは、new NYCheesePizza()やnew ChicagoCheesePizza()を直接使用しない

Pizza pizza = createPizza(type);だけを使用する

実際にどのPizzaを生成するかはNYPizzaStoreとChicagoPizzaStoreが決める